

3.4 Pointer Arithmetic

The bishop asked Sam to deliver a stack of flyers to all the apartments in his complex. Remembering that an apartment address is like a pointer (where the dereference operator takes you to the apartment) and the apartment block is like an array (a contiguous set of addresses referenced by the first address), Sam realizes that this is a pointer arithmetic problem. He starts at the first address (the pointer to the first item in the array), delivers the flyer (dereferences the pointer with the `*`), and increments the address (adds one to the pointer with `p++`). This is continued (with a FOR loop) until the last address is reached.

Objectives

By the end of this class, you will be able to:

- Understand how to advance a pointer variable with the `++` operator.
- Traverse a string with the second standard FOR loop.
- Explain the difference between a constant pointer and a pointer to a constant value.

Prerequisites

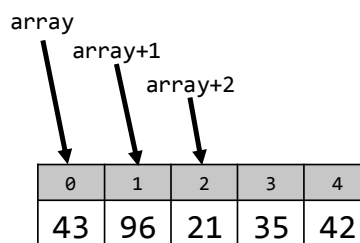
Before reading this section, please make sure you are able to:

- Declare a pointer variable (Chapter 3.3).
- Point to an existing variable in memory with a pointer variable (Chapter 3.3).
- Get the data out of a pointer (Chapter 3.3).
- Pass a pointer to a function (Chapter 3.3).
- Understand the role the null character plays in string definitions (Chapter 3.2).
- Write a loop to traverse a string (Chapter 3.2).

Overview

Pointer arithmetic is the process of manipulating pointers to better facilitate accessing arrays of data in memory. Though the term “arithmetic” implies that a whole range of arithmetic operations can be performed, we are normally restricted to addition and subtraction.

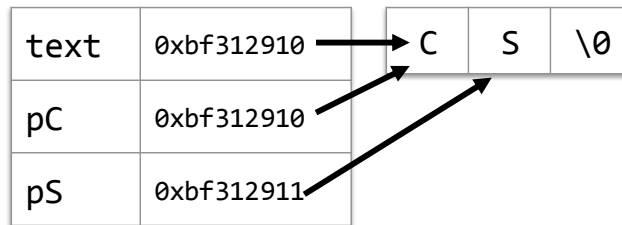
Recall from Chapter 3.3 that a pointer is a reference to a location in memory. We typically do not know where this memory is located until run-time; the operating system places the program in memory and often puts it in a different location every time. Recall from Chapter 3.0 that arrays are collections of data guaranteed to reside in contiguous blocks of memory. From these two observations it should be clear that, given some `array[i]`, the location `array[i + 1]` should be in the adjacent location in memory. Pointer arithmetic is the process of leveraging this proximity to access array data.



Consider the following code:

```
{
    char text[] = "CS";    // some buffer in memory
    char * pC = text;      // text refers to the first item, so pC does as well
    char * pS = text + 1;  // the next location in memory is one item beyond the first
}
```

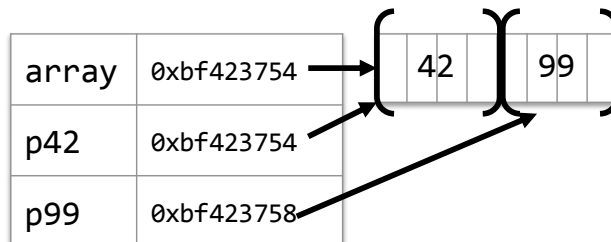
In this example, `text` points to the string "CS" or, more explicitly, to the first letter of the string. Next, the variable `pC` inherits its address from `text` which points to 'C' (hence the name `pC`). Finally, since the letter 'S' is one letter beyond 'C' in the string, it follows it has an address one greater than the 'C'. Thus, we can find the address of 'S' by taking the address of 'C' (as specified by the variable `text`) and adding one:



It turns out that integer pointers work the same way. The only difference is that integers are 4 bytes in length where characters are one. However, the pointer arithmetic is the same:

```
{
    int array[] =
    {
        42, 99
    };
    int * p42 = array;      // just like characters, points to the first item
    int * p99 = array + 1;  // add one to move forward four bytes!
}
```

Just like the first example, `array` points to the first number in the list. Next, the variable `p42` has the same value (the address of 42) as `array`. Finally, since the number 99 is next to the number 42, it follows it will have an address one greater. Thus, we can find the address of 99 by taking the address of 42 and adding one.



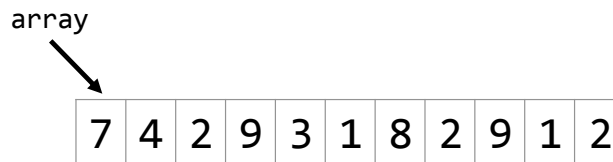
Because arrays (including strings) are just pointers, it is often most convenient to traverse an array with a pointer than with an index. This involves incrementing the pointer variable rather than an index.

Arrays

As discussed before, arrays are simply pointers. This gives us two different notations for working with arrays: the square bracket notation and the star notation. Consider the following array:

```
int array[] =  
{  
    7, 4, 2, 9, 3, 1, 8, 2, 9, 1, 2  
};
```

This can be represented with the following table:



Consider the first element in an array. We can access this item two ways:

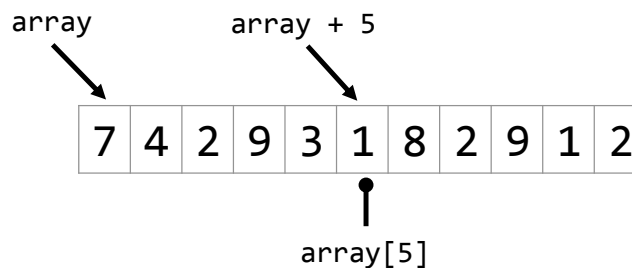
```
cout << "array[0] == " << array[0] << endl;  
cout << "*array == " << *array << endl;  
assert(array[0] == *array);
```

The first output line will of course display the value 7. We learned this from Chapter 3.0. The second will dereference the array pointer, yielding the value it points to. Since pointers to arrays always point to the first item, this too will give us the value 7. In other words, there is no difference between `*array` and `array[0]`; they are the same thing!

Similarly, consider the 6th item in the list. We can access with:

```
cout << "array[5] == " << array[5] << endl;  
cout << "*(array + 5) == " << *(array + 5) << endl;  
assert(array[5] == *(array + 5));
```

This is somewhat more complicated. We know the 6th item in the list can be accessed with `array[5]` (remembering that we start counting with zero instead of one). The next statement (with `*(array + 5)` instead of `array[5]`) may be counterintuitive. Since we can point to the 6th item on the list by adding five to the base pointer (`array + 5`), then by dereferencing the resulting pointer we get the data:



Therefore we can access any member of an array using either the square bracket notation or the star-parentheses notation.



It turns out that the square bracket array notation (`[]`) is actually a macro expansion for an addition and a dereference. Consider the following code:

```
cout << array[5] << endl;
```

We have already discussed how this is the same as adding five to the base pointer and dereferencing:

```
cout << *(array + 5) << endl;
```

Now, from the commutative property of addition, we should be able to re-order the operands of an addition without changing the value:

```
cout << *(5 + array) << endl;
```

From here, it gets a bit dicey. The claim is that the addition and dereference operator combined are the same as the square bracket operator. If this is true (and it is!), then the following should be true:

```
cout << 5[array] << endl;
```

Thus, under the covers, arrays are just pointers. The square brackets are just used to make it more intuitive and easy to understand for novice programmers. Please see the following code for an example of how this works at [3-4-notationAbuse.cpp](#) or:

```
/home/cs124/examples/3-4-notationAbuse.cpp
```

Pointers as Loop Variables

Up to this point, all the loops we have written to access individual members of a string or array have used index variables and the square-bracket notation. It turns out that we can write an equivalent pointer-loop for each index-loop. These loops tend to perform better than their index counterparts because fewer assembly instructions are required. The two main applications for pointers as loop variables are array traversing loops and string traversing loops.

Array traversing loop

Recall that the standard way to use a for loop to walk through an array is:

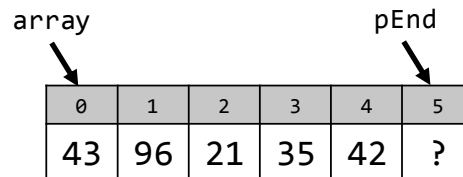
```
for (int i = 0; i < num; i++)
    cout << array[i];
```

It turns out we can use a pointer to loop through an array of integers if the length of the array is known.

Consider the following array:

```
int array[] =
{
    43, 96, 21, 35, 42
};
```

In this example, the pointer to the beginning of the list is `array` and the pointer to the item off the end of the list is `array + num`:



This allows us to write a loop to walk through the list:

```
int * pEnd = array + num;
for (int * p = array; p < pEnd; p++)
    cout << *p << endl;
```

Observe how, with each iteration, the pointer variable `p` advances by one address. This continues until `p` is no longer less-than the item off the end of the list `pEnd`. Since we are working with arrays, we can dereference each item in the list with `*p`. For an example of this loop in action, please see [3-4-pointerArray.cpp](#) or:

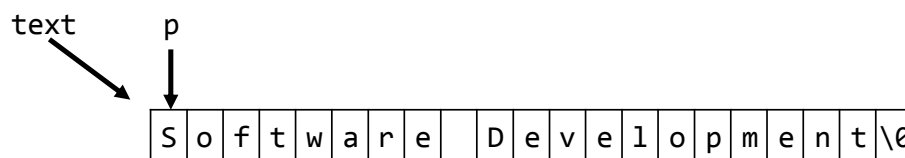
```
/home/cs124/examples/3-4-pointerArray.cpp
```

String traversing loop

With strings, the end of the string is defined as the null-character. This leads us to the second standard for loop: traversing a string with a pointer.

```
for (char * p = text; *p; p++)
    cout << *p;
```

Just like with the aforementioned array example, we advance the pointer rather than an index into the string. The big difference is the controlling Boolean expression: a null-terminator check rather than looking for a pointer to the end of the string.



In this example, `text` points to the first item in the string ('S'). The loop starts by assigning `p` to also point to the first item. The loop continues by advancing `p` through the string. The loop terminates when `*p` is no longer true. This occurs when `p` points to the null-character.

Example 3.4 – String Traverse

Demo

This example will demonstrate how to traverse a string using the pointer notation. It will use the second standard FOR loop:

```
for (char * p = text; *p; p++)  
    ;
```

Problem

Write a function to display the contents of a string, one character on each line:

```
Please enter the text: Point  
P  
o  
i  
n  
t
```

Solution

The function will take a pointer to a character as a parameter. Recall that arrays are pointers to the first item in the list. The same thing is true with strings. String variables are actually pointers to the first character. Thus the prototype is:

```
void display(const char * text);
```

From here, we will use the second standard FOR loop to iterate through each item in the string. Recall that the dereference operator * is needed to retrieve the data from the string.

```
/* *****  
 * DISPLAY  
 * Display the passed text one character  
 * at a time using a pointer  
 * ***** */  
void display(const char * text)  
{  
    // second standard for loop  
    for (const char *p = text;          // p will point to each item in the string  
         *p;                          // as long as *p is not the NULL character  
         p++;                          // increment one character at a time  
        {  
        cout << '\t' << *p << endl;    // access each item with the dereference *  
        }  
}
```

Challenge

As a challenge, can you modify the program so the function displays every other character in the string? What happens when there are an odd number of characters in the string? How can you detect that condition so the program does not malfunction?

See Also

The complete solution is available at [3-4-traverse.cpp](#) or:

```
/home/cs124/examples/3-4-traverse.cpp
```



Example 3.4 – Convert Case

Demo	This example will demonstrate how to walk through a loop using the pointer notation. In this case, processing is performed on every character in the string.				
Problem	Write a program to convert uppercase characters to lowercase and vice-versa. Please enter your text: <u>SOFTWARE eNGINEERING</u> Software Engineering				
Solution	There are two components to this problem. The first is to use the second standard FOR loop to walk through the string. The second part is to use the pointer notation instead of the array notation to reference each member of the string. <table border="1"> <thead> <tr> <th>Array notation</th><th>Pointer notation</th></tr> </thead> <tbody> <tr> <td> <pre>void convert(char text[]) { for (int i = 0; text[i]; i++) if (isupper(text[i])) text[i] = tolower(text[i]); else text[i] = toupper(text[i]); }</pre> </td><td> <pre>void convert(char * text) { for (char * p = text; *p; p++) if (isupper(*p)) *p = tolower(*p); else *p = toupper(*p); }</pre> </td></tr> </tbody> </table> Notice how the array notation uses the square bracket [] notation to declare the function parameter while the pointer notation uses the *. Both notations mean mostly the same thing, “a pointer to the first item in the string.” In the array notation solution, the individual items in the string are referenced with text[i]. With the pointer notation, we use the *p notation. Both produce the same results but using a different mechanism. In general, the pointer notation is preferred because it is simpler and more efficient.	Array notation	Pointer notation	<pre>void convert(char text[]) { for (int i = 0; text[i]; i++) if (isupper(text[i])) text[i] = tolower(text[i]); else text[i] = toupper(text[i]); }</pre>	<pre>void convert(char * text) { for (char * p = text; *p; p++) if (isupper(*p)) *p = tolower(*p); else *p = toupper(*p); }</pre>
Array notation	Pointer notation				
<pre>void convert(char text[]) { for (int i = 0; text[i]; i++) if (isupper(text[i])) text[i] = tolower(text[i]); else text[i] = toupper(text[i]); }</pre>	<pre>void convert(char * text) { for (char * p = text; *p; p++) if (isupper(*p)) *p = tolower(*p); else *p = toupper(*p); }</pre>				
Challenge	As a challenge, try to modify the above program to count the number of uppercase letters in the input stream. The prototype is: int countUpper(const char *text);				
See Also	The complete solution is available at 3-4-toggleCase.cpp or: /home/cs124/examples/3-4-toggleCase.cpp 				

Example 3.4 – String Length

Demo

This example will demonstrate how to walk through a string using the pointer notation.

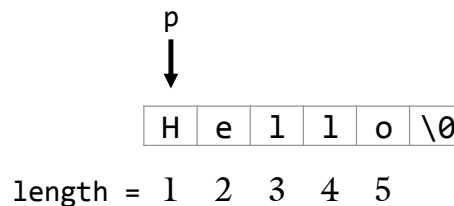
Problem

Write a program to find the length of a string. This will be done using the array notation, the pointer notation using the second standard FOR loop, and the optimal solution:

```
Please enter your text: Software
Array notation:      8
Pointer notation:    8
Optimal version:     8
```

Solution

The most straight-forward way to do this is to walk through the string using the second standard FOR loop. Here, the pointer `p` will serve as the loop control variable. Inside the body of the loop, we will have a `length` variable incrementing with each iteration.



Since we only increment `length` when we have not yet encountered the null character, the value at the end of the loop should be the length of the string.

```
/******
 * STRING LENGTH : pointer version
 * Increment the length variable with
 * every iteration
 *****/
int stringLength(char * text)
{
    int length = 0;                // declared out of FOR loop scope

    for (char * p = text; *p; p++) // second standard FOR loop
        length++;                 // increment length every iteration

    return length;                 // return the length variable
}
```

Notice how two variables (`length` and `p`) increment with each iteration of the loop. A more efficient solution would not have this redundancy. Please see the video and associated code for the better solution.

See Also

The complete solution is available at [3-4-stringLength.cpp](#) or:

```
/home/cs124/examples/3-4-stringLength.cpp
```



Example 3.4 – String Compare

Demo

This example will demonstrate how to iterate through two strings at the same time. This will be done using the array notation with an index, the pointer notation using the second standard FOR loop, and an optimal solution.

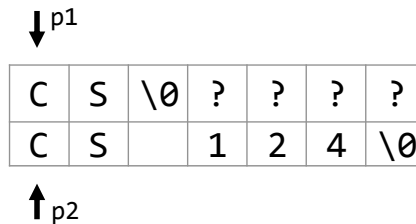
Problem

Write a program to determine if two strings are the same. Note that the equivalence operator alone is insufficient (`text1 == text2`) because it will just compare two addresses. We also cannot use the dereference operator alone (`*text1 == *text2`) because it will just compare the first two items in the string. It will be necessary to compare each item in both strings using a loop.

```
Please enter text 1: Software
Please enter text 2: Software
    Array notation:   Equal
    Pointer notation: Equal
    Optimal version:  Equal
```

Solution

To solve the problem, it is necessary to have two pointer variables moving in tandem through the two strings:



The hardest problem here is to know when to terminate the loop. There are three conditions that could terminate the loop:

1. When `*p1 != *p2` or when the currently considered character is different.
2. When the end of the first string is reached. `*p1 == '\0'`
3. When the end of the second string is reached. `*p2 == '\0'`

Thus, when either of these conditions are met, the loop terminates. If the first condition terminates the loop, then the strings are different. Otherwise, the strings are the same.

```
bool isEqual(char * text1, char * text2)
{
    char * p1;                // for text1
    char * p2;                // for text2

    for (p1 = text1, p2 = text2;    // two pointers require two initializations
        *p1 == *p2 &&            // same logic as with array index version but
        *p1 && *p2;                // somewhat more efficient
        p1++, p2++)                // both pointers need to be advanced
    ;

    return (*p1 == *p2);
}
```

See Also

The complete solution is available at [3-4-stringCompare.cpp](#) or:

```
/home/cs124/examples/3-4-compareString.cpp
```



Constant Pointers

Earlier in the semester, we introduced the notion of the `const` modifier:

```
const int SIZE = 10;           // SIZE can never change
```

The `const` modifier allows the programmer to specify (and the compiler to enforce!) that a given variable will never change. Along with this, we can also point to constant data. The most common time we do this is when passing arrays as parameters:

```
void displayName(const char * name);    // displayName() cannot change name
```

In this example, the function `displayName()` is not able to change any of the data in the string `name`.

Another class of constant data is a constant pointer. A constant pointer refers to a pointer that must always refer to a single location in memory. Consider the following code:

```
int array[4];           // declare an array of integers
array[0] = 6;           // legal. The data is not constant.
array++;               // ERROR! We cannot change the variable 'array'!
```

When we declare an array, the array pointer will always refer to the same location of memory. We cannot perform pointer arithmetic on this variable.

It turns out that whenever we declare an array using the square bracket notation (`[]`), whether in a function parameter or as a local variable, that variable is a constant pointer. Consider the following code:

```
void function(      char * parameter1,    // can change both pointer and value
                   const char * parameter2, // can change pointer but not value
                   char  parameter3[],    // can change value but not pointer
                   const char parameter4[]) // can change neither value nor pointer
{
    // change the value
    *parameter1 = 'x';           // legal because parameter 1 is not a const
    *parameter2 = 'x';           // ERROR! parameter2 is a const!
    *parameter3 = 'x';           // legal because parameter 3 is not a const
    *parameter4 = 'x';           // ERROR! parameter4 is a const

    // change the pointer
    parameter1++;                // legal; not a const pointer
    parameter2++;                // legal; not a const pointer
    parameter3++;                // ERROR! parameter3 is a const pointer
    parameter4++;                // ERROR! parameter4 is a const pointer
}
```

When a parameter is passed with the square brackets or when an array is declared in a function, it is still possible to do pointer arithmetic. Consider the following example:

```
{
    int array[4];    // constant pointer.
    int * p = array; // not a constant pointer!

    p++;             // we cannot do this with array. array++ would be illegal!
}
```

Even though `array` and `p` refer to the same location in memory, we can increment `p` but not `array` because `array` is a constant pointer whereas `p` is not.



The most common way to make a parameter a constant pointer is to use the [] notation in the parameter declaration. This, unfortunately, implies that the pointer is an array (which it may not be!). Another way to make a variable a constant pointer is to use the const modifier after the *:

```
void function(    int *           pointer,
                const int *       pointerToConstant,
                int * const constantPointer,
                const int * const constantPointerToConstant);
```

This is the same as:

```
void function(    int *           pointer,
                const int *       pointerToConstant,
                int               constantPointer[],           // note the []s
                const int         constantPointerToConstant[]); // more []s
```

Problem 1

What is the output of the following code?

```
{
    char * a;
    char * b = a;
    char  c = 'x';
    char  d = 'y';
    b = &c;
    a = &d;
    *b = 'z';
    *a = *b;
    cout << "d == " << d << endl;
}
```

a	b	c	d

Answer:

Please see page 258 for a hint.

Problem 2

How much memory does each of the following reserve?

char text[2];

char text[] = "CS 124";

char text[] = "Point";

char * text[6];

Please see page 216 for a hint.

Problem 3

What is the output of the following code?

```
{
    char x[] = "Sam";
    char y[] = "Sue";
    char * z;

    if (x == y)
        z = x;
    else
        z = y;

    cout << *z << endl;
}
```

Answer:

Please see page 277 for a hint.

Problem 4

What is the output of the following code?

```
{
    char text[] = "Software";

    for (int i = 4;
        i < 7;
        i++)
        cout << text[i];

    cout << endl;
}
```

Answer:

Please see page 219 for a hint.

Problem 5

What is the output of the following code?

```
{
    char a1[10] = "42";
    char a2[10];

    int i;
    for (i = 0; a1[i]; i++)
        a2[i] = a1[i];
    a2[i] = '\0';

    cout << a2 << endl;
}
```

Answer:

Please see page 277 for a hint.

Problem 6

What is the output of the following code?

```
{
    char text[] = "Banana";

    char * pA = text + 2;

    cout << *pA << endl;
}
```

Answer:

Please see page 148 for a hint.

Problem 7

What is the output of the following code?

```
{
    int array[] = {1, 2, 3, 4};

    cout << *(array + 2) << endl;
    cout << array + 2 << endl;
    cout << array[3] << endl;
    cout << *array + 3 << endl;
}
```

Answer:

Please see page 271 for a hint.

Problem 8

What is the output of the following code?

```
{
    char * a = "Software";
    char * b;

    while (*a)
        b = a++;

    cout << *b << endl;
}
```

Answer:

Please see page 49 for a hint.

Problem 9

What is the output of the following code?

```
{
    char text[] = "Software";

    int a = 0;
    for (char * p = text; *p; p++)
        a++;

    cout << a << endl;
}
```

Answer:

Please see page 49 for a hint.

Assignment 3.4

Start with Assignment 3.2. Modify `countLetters()` so that it walks through the string using pointers instead of array indexes. The output should be exactly the same as with Assignment 3.2.

Example

Two examples... The user input is underlined.

Example 1:

```
Enter a letter: z  
Enter text: NoZ'sHere!  
Number of 'z's: 0
```

Example 2:

```
Enter a letter: a  
Enter text: Brigham Young University - Idaho  
Number of 'a's: 2
```

Assignment

The test bed is available at:

```
testBed cs124/assign34 assignment34.cpp
```

Don't forget to submit your assignment with the name "Assignment 34" in the header.

Please see page 261 for a hint.